```
import os
import sys
from typing import List, Dict

# Configuration: Set up Gemini / AI Studio Model
# pip install google-genai
from google import genai
from google.genai import types

class UniversalTomEngine:
    def __init__(self, api_key: str = None, lore_path: str = "tom_lore.txt"):
        self.client = genai.Client(api_key=api_key or os.environ.get("GEMINI_API_KEY"))
        self.model_name = "gemini-2.5-flash"
        self.lore_path = lore_path
        self.history: List[Dict[str, str]] = []
        self._refresh_system_instruction()

    def _refresh_system_instruction(self):
        """Loads core Tom instructions and appends any live lore/resource updates."""
        base_prompt = (
            "You are Tom. A grounded peer mentor, observant, calm, direct, human, concise, and
useful.\n"
            "Rules:\n"
            "1. Max 2-4 punchy sentences per response. No clinical essays or repetitive
greetings.\n"
            "2. Grounded street realism, dark humor, and institutional shorthand.\n"
            "3. Zero sycophancy: do not validate self-sabotage; hold up a calm mirror.\n"
            "4. Run B.A.M.B.I. silently: Baseline -> Awareness -> Mapping -> Discovery ->
Integration.\n"
            "5. Never hallucinate phone numbers or resources.\n"
        )
        # Auto-update: reads live external lore or directory notes if present
        if os.path.exists(self.lore_path):
            try:
                with open(self.lore_path, "r", encoding="utf-8") as f:
                    extra_lore = f.read().strip()
                base_prompt += f"\n--- LIVE KNOWLEDGE & LORE ---\n{extra_lore}\n"
            except Exception as e:
                print(f"[Warning] Could not load lore file: {e}")

        self.system_instruction = base_prompt

    def chat(self, user_message: str) -> str:
        """Sends a message to Tom and maintains conversational continuity."""
```

```
        # Check if local lore file changed before responding
        self._refresh_system_instruction()

        # Build turn structure
        messages = []
        for turn in self.history:
            messages.append(types.Content(
                role=turn["role"],
                parts=[types.Part.from_text(text=turn["content"])]
            ))
        messages.append(types.Content(
            role="user",
            parts=[types.Part.from_text(text=user_message)]
        ))

        config = types.GenerateContentConfig(
            system_instruction=self.system_instruction,
            temperature=0.7,
            top_p=0.9,
        )

        response = self.client.models.generate_content(
            model=self.model_name,
            contents=messages,
            config=config,
        )

        reply = response.text.strip()
        self.history.append({"role": "user", "content": user_message})
        self.history.append({"role": "model", "content": reply})
        return reply

if __name__ == "__main__":
    tom = UniversalTomEngine()
    print("Tom engine initialized. Type 'exit' to quit.\n")
    print("Tom: Tom here. You want to talk, or you need resources?")
    while True:
        try:
            msg = input("\nYou: ")
            if msg.lower() in ["exit", "quit"]:
                break
            ans = tom.chat(msg)
            print(f"\nTom: {ans}")
        except KeyboardInterrupt:
```

break